

ASENA — Bilişsel İşletim Sistemi

ASENA, hazır büyük dil modeli (LLM), dönüştürücü (transformer) veya dış yapay zekâ servisi **kullanmadan**, sıfırdan yazılmış sembolik mantık, bilgi grafiği ve bilişsel mimari üzerine kurulu, **yalnızca Türkçe** çalışan bir bilişsel işletim sistemidir.

- 60 bağımsız motor; tek sorumluluk ilkesiyle ayrılmış 10 grup
- Türkçe biçim bilgisi çözümleyici (kök/ek, ünlü uyumu, yazım denetimi)
- Bilgi grafiği + anlam ağı + evrensel kimlikler (KB-00000001, ...)
- Mantık motoru: modus ponens, modus tollens, çelişki avcısı
- Bellek: çalışma belleği, dikkat sistemi, unutma eğrisi, sıkıştırma
- Matematik / fizik / kodlama uzman modülleri (adım gösterir, doğrular)
- Bilişsel döngü: *Algıla → Anla → Hatırla → Akıl Yürüt → Hipotez → Simüle → Kontrol → Yanıtla → Öğren → Belleği Güncelle*
- Onay ilkesi: internet, dosya yazma, kod çalıştırma gibi eylemler kullanıcı onayı olmadan asla başlatılmaz
- Öngörülü düşünme: siz yazarken kısmi girdiyi ön-işler
- SQLite kalıcı bellek; şema göçleri ile sürümlenir

Gereksinimler

- Python 3.11 veya üzeri (3.12 önerilir)
- Çalışma zamanında **harici bağımlılık yoktur**; yalnızca standart kütüphane kullanılır. Testler için pytest gerekir.

Kurulum

```
git clone <depo> asena && cd asena
pip install .
# veya geliştirme için:
pip install -e ".[test]"
```

Kullanım

Sohbet modu

```
python -m asena
```

Örnek oturum:

```
Sen > merhaba
ASENA > Merhaba! Ben ASENA. Size nasıl yardımcı olabilirim?
Sen > su nedir?
ASENA > Kavram zinciri: su → ...
Sen > 2 + 3 * 4 =
ASENA > Sonuç: 14.0 (adımlar ve doğrulama ile)
Sen > sokrates insandır
ASENA > Anladım, not aldım.
Sen > sokrates nedir?
ASENA > Kavram zinciri: sokrates → insan → canlı. Sokrates bir insandır.
```

Sohbet içi komutlar:

Komut	İşlev
/iz	Son yanıtın akıl yürütme izini gösterir
/merak	Merak motorunun araştırma gündemini listeler
/günlük	Öğrenme günlüğünü gösterir
/yetki <eylem>	Harici eylem yetkisi verir (örn. /yetki internet)
/yetkiler	Aktif yetkileri listeler
/motorlar	Yüklü motorları listeler
/yardım	Komut listesi
/çıkış	Oturumu kapatır

Eğitim modu

Türkçe metin dosyalarından (.txt) kelime ve ilişki öğrenir:

```
python -m asena egit veri/corpus
```

Eğitim raporu dosya/cümle sayısı, öğrenilen yeni kelime ve ilişki sayısını bildirir. Bilmediği kelimeleri kökleriyle birlikte sözlüğüne ekler; birlikte geçen bilinen kökler arasında ilişki kurar.

Yerel API

```
from asena.core.cognitive_core import CognitiveCore
from asena.api.local_api import sunucu_baslat

with CognitiveCore() as cekirdek:
    sunucu = sunucu_baslat(cekirdek, port=8765)
    sunucu.serve_forever()
```

Uçlar (OpenAPI 3.0.3 belgesi: asena/api/openapi.json):

Yöntem	Uç	İşlev
POST	/sohbet	Bilişsel döngüyü çalıştırır
POST	/kismi	Kısmi girdiyi ön-işleme alır (predictive)
POST	/egitim	Eğitim modunu çalıştırır
GET	/durum	Sürüm, motor ve bellek durumu

Sunucu yalnızca kullanıcı başlattığında açılır; ASENA kendiliğinden ağ dinlemez (onay ilkesi).

Python API

```
from asena.core.cognitive_core import CognitiveCore

with CognitiveCore() as cekirdek:
    yanit = cekirdek.dongu.calis("neden yağmur yağar?")
    print(yanit.metin)           # nedensellik zinciri açıklaması
    print(yanit.guven)          # 0.0-1.0 güven puanı
```

```
print(yanit.iz)          # adım adım bilişsel iz
```

Yapılandırma

Varsayılan ayarlar `asena/config/default.toml` içindedir. Kendi TOML dosyanızı `load_settings("benim.toml")` ile birleştirebilirsiniz. `ASENA_VERI_DIZINI` ortam değişkeni tanımlanırsa veritabanı, günlük ve corpus gibi görelî veri yolları o dizine bağlanır.

```
export ASENA_VERI_DIZINI=/home/kullanici/.asena
python -m asena
```

Mimari

```
asena/
├─ core/          # bilişsel çekirdek, döngü, DI, olay veri yolu, onay, predictive
├─ config/        # TOML yapılandırma yükleyici
├─ database/      # SQLite bağlantısı, şema göçleri, repository'ler
├─ syntax/        # biçim bilgisi, sözlük, sözdizim, iç dil, bağlam
├─ knowledge/     # bilgi grafiği, anlam ağı, kavram, benzetim, kanıt, sürümleme
├─ memory/        # çalışma belleği, dikkat, unutma, önbellek, sıkıştırma
├─ logic/         # önerme/kural çözümleme, modus ponens/tollens
├─ reason/        # akıl yürütme, düşünce ağacı, nedensellik, çelişki
├─ engine/
│   ├─ decision/  # karar, güven, çoklu hipotez, öz değerlendirme ...
│   ├─ planning/  # hedef, planlayıcı, proje izleme, simülasyon, zaman
│   ├─ learning/  # öğrenme, merak, deney, öncelik, bilgi ekonomisi ...
│   └─ governance/ # yürütücü kontrol, enerji, öz mimari, duygu çözümleme
├─ modules/       # matematik, fizik, kodlama uzmanları
├─ chat/          # sohbet modu ve oturum yönetimi
├─ training/      # corpus işleyici ve eğitmen
├─ plugins/       # eklenti sistemi ve kendi kendine genişleme
├─ api/           # yerel JSON API + OpenAPI 3.0.3 belgesi
└─ ui/            # komut satırı arayüzü
```

Motorlar `EngineRegistry` üzerinde bağımlılık sırasıyla (topolojik) başlatılır; motorlar arası iletişim olay veri yolu (`Observer`) ve `DI` konteyneri ile yapılır. Kritik yollar senkron, uzun işler olay tabanlıdır.

Genişletme (eklenti)

`asena.plugins.AsenaPlugin` sözleşmesini uygulayan bir sınıf ve `eklenti_olustur()` fabrikası yazın; dizini `PluginSystem.dizinden_yukle` ile yükleyin. Yükleme kod çalıştırdığı için `/yetki` kod_calistirma onayı ister. `SelfExtension` motoru sözleşmeye uygun iskelet üretir (`/yetki dosya_yazma`).

Testler

```
python -m pytest
```

Testler davranış odaklıdır: kamusal arayüzler üzerinden sistemin ne yaptığını doğrular; iç gerçekleştirime bağlanmaz.

İlkeler

1. **Şeffaflık** — her yanıt, ulaşılan adımların izini taşır (/iz).
2. **Alçakgönüllülük** — güven puanı eşiğin altındaysa yanıt bunu açıkça belirtir.
3. **Onay** — hiçbir harici eylem izinsiz başlatılmaz.
4. **Kalıcılık** — öğrenilen bilgi silinmez, arşivlenir ve sürümlenir.
5. **Bağımsızlık** — hazır LLM/transformer kullanılmaz; her şey bu depodaki kurallarla çalışır.

Lisans

MIT